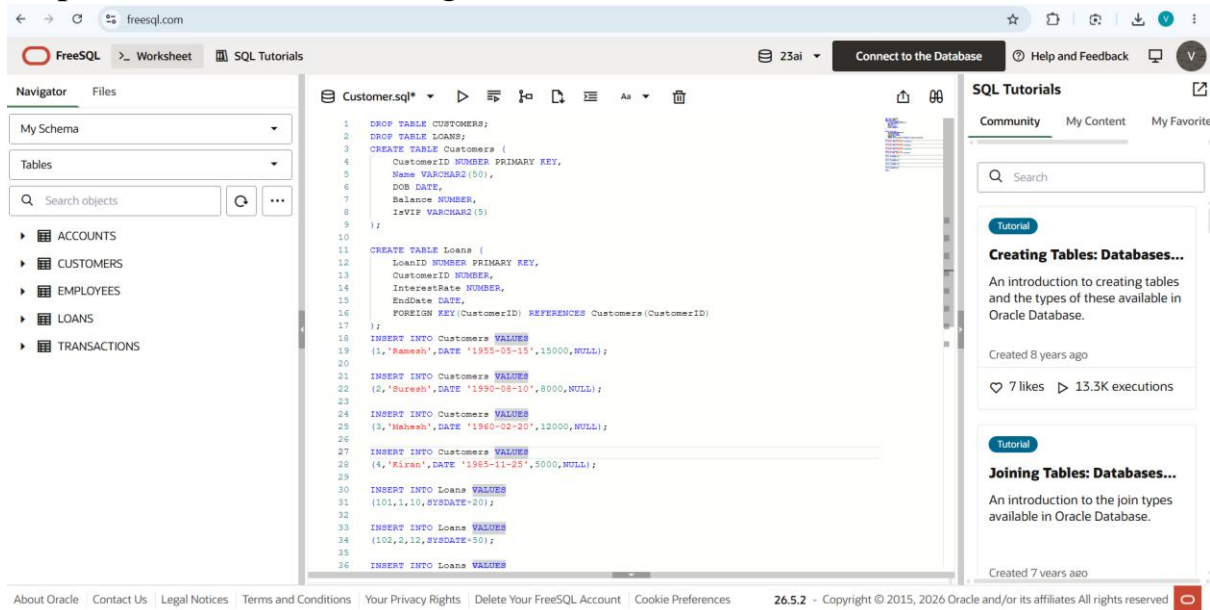


Exercise 1: Control Structures in PL/SQL

Step 1: Table created values get inserted



The screenshot displays the FreeSQL online SQL editor interface. The main editor window shows a PL/SQL script for creating and inserting data into two tables: Customers and Loans. The script is as follows:

```
1 DROP TABLE CUSTOMERS;
2 DROP TABLE LOANS;
3 CREATE TABLE Customers (
4   CustomerID NUMBER PRIMARY KEY,
5   Name VARCHAR2(50),
6   DOB DATE,
7   Balance NUMBER,
8   IsVIP VARCHAR2(5)
9 );
10
11 CREATE TABLE Loans (
12   LoanID NUMBER PRIMARY KEY,
13   CustomerID NUMBER,
14   InterestRate NUMBER,
15   EndDate DATE,
16   FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)
17 );
18
19 INSERT INTO Customers VALUES
20 (1, 'Ramesh', DATE '1955-05-15', 15000, NULL);
21
22 INSERT INTO Customers VALUES
23 (2, 'Suresh', DATE '1990-08-10', 8000, NULL);
24
25 INSERT INTO Customers VALUES
26 (3, 'Mahesh', DATE '1960-02-20', 12000, NULL);
27
28 INSERT INTO Customers VALUES
29 (4, 'Kiran', DATE '1985-11-25', 5000, NULL);
30
31 INSERT INTO Loans VALUES
32 (101, 1, 10, SYSDATE-20);
33
34 INSERT INTO Loans VALUES
35 (102, 2, 12, SYSDATE-50);
36
37 INSERT INTO Loans VALUES
```

The interface includes a Navigator panel on the left showing a schema named 'My Schema' with tables 'ACCOUNTS', 'CUSTOMERS', 'EMPLOYEES', 'LOANS', and 'TRANSACTIONS'. The right sidebar contains 'SQL Tutorials' with links to 'Community', 'My Content', and 'My Favorite'. The bottom status bar indicates the version is 26.5.2 and includes copyright information for Oracle.

Step 2:

Scenario 1

Problem Statement:

Apply a 1% discount to loan interest rates for customers above 60 years old.

Explanation:

This PL/SQL block retrieves all customers from the Customers table and calculates their age using the DOB (Date of Birth) field. It then loops through each customer record and checks whether the customer's age is greater than 60 years. If the condition is satisfied, the corresponding loan's interest rate in the Loans table is reduced by 1%. After processing all customers, the COMMIT statement permanently saves the updated interest rates in the database. A success message is displayed using DBMS_OUTPUT.PUT_LINE to confirm that the discount has been applied successfully.

```

1  SELECT * FROM Loans;
2
3  DECLARE
4      v_age NUMBER;
5  BEGIN
6
7      FOR cust IN (
8          SELECT CustomerID, DOB
9          FROM Customers
10         ) LOOP
11
12             v_age := FLOOR(
13                 MONTHS_BETWEEN(
14                     SYSDATE,
15                     cust.DOB
16                 ) / 12);
17
18             IF v_age > 60 THEN
19
20                 UPDATE Loans
21                 SET InterestRate =
22                     InterestRate - 1
23                 WHERE CustomerID =
24                     cust.CustomerID;
25
26             END IF;
27
28         END LOOP;
29
30     COMMIT;
31
32     DBMS_OUTPUT.PUT_LINE(
33         'Interest rate discount applied successfully. ');
34
35 END;

```

OUTPUT:

	LOANID	CUSTOMERID	INTERESTRATE	ENDDATE
1	101	1	10	7/12/2026, 1:42:20
2	102	2	12	8/11/2026, 1:42:20
3	103	3	11	7/2/2026, 1:42:20 P
4	104	4	9	9/20/2026, 1:42:20

Step 3:

Scenario 2

Problem Statement:

Set **IsVIP** = **TRUE** for customers whose balance is greater than 10,000.

Explanation:

This PL/SQL block retrieves all customer records from the **Customers** table

and checks each customer's account balance. It loops through the records one by one using a **FOR LOOP**. If a customer's balance is greater than **10,000**, the **IsVIP** column is updated to **TRUE**; otherwise, it is updated to **FALSE**. After processing all customer records, the **COMMIT** statement permanently saves the changes in the database. A success message is displayed using **DBMS_OUTPUT.PUT_LINE** to indicate that the VIP status has been updated successfully.

```

1  SELECT * FROM Customers;
2
3  BEGIN
4
5      FOR cust IN (
6          SELECT CustomerID,
7                  Balance
8          FROM Customers
9      ) LOOP
10
11          IF cust.Balance > 10000 THEN
12
13              UPDATE Customers
14              SET IsVIP='TRUE'
15              WHERE CustomerID=
16                  | cust.CustomerID;
17
18          ELSE
19
20              UPDATE Customers
21              SET IsVIP='FALSE'
22              WHERE CustomerID=
23                  | cust.CustomerID;
24
25          END IF;
26
27      END LOOP;
28
29      COMMIT;
30
31      DBMS_OUTPUT.PUT_LINE(
32          'VIP status updated successfully.');
```

Output:

	CUSTOMERID	NAME	DOB	BALANCE	ISVIP
1	1	Ramesh	5/15/1955, 12:00:00	15000	TRUE
2	2	Suresh	8/10/1990, 12:00:00	8000	FALSE
3	3	Mahesh	2/20/1960, 12:00:00	12000	TRUE
4	4	Kiran	11/25/1985, 12:00:00	5000	FALSE

Step 4:

Scenario 3:

Problem Statement:

Send reminder messages for loans that are due within the next 30 days.

Explanation:

This PL/SQL block retrieves loan records whose EndDate falls between the current date (SYSDATE) and the next 30 days. It joins the Customers and Loans tables to obtain customer names along with their loan details. The block then loops through each matching record and displays a reminder message using DBMS_OUTPUT.PUT_LINE, showing the customer's name, loan ID, and due date. This helps the bank proactively notify customers about upcoming loan repayments and avoid missed due dates.

```
1  SELECT
2      c.Name,
3      l.LoanID,
4      TO_CHAR(l.EndDate, 'DD-MON-YYYY') AS Due_Date
5  FROM Customers c
6  JOIN Loans l
7  ON c.CustomerID = l.CustomerID
8  WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;
```

Output:

	NAME	LOANID	DUE_DATE
1	Ramesh	101	08-JUL-2026
2	Mahesh	103	02-JUL-2026
		103	

Exercise 3 – Stored Procedures in PL/SQL

Step 1: Scenario 1

Problem Statement

Calculate and credit 1% monthly interest to all savings accounts.

Explanation:

This stored procedure automatically calculates monthly interest for every account present in the SavingsAccounts table. The balance of each account is increased by 1% of its existing value using an UPDATE statement. Once all account balances are updated, the COMMIT command saves the modifications permanently. A confirmation message is displayed after successful execution, indicating that the monthly interest has been credited.

```
1  CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2  AS
3  BEGIN
4
5      UPDATE Accounts
6      SET Balance = Balance + (Balance * 0.01)
7      WHERE AccountType = 'Savings';
8
9      COMMIT;
10
11 END;
12 /
13 BEGIN
14     ProcessMonthlyInterest;
15 END;
16 /
17 SELECT * FROM Accounts;
18
```

Output:

Query result Script output DBMS output Explain Plan SQL history						
  Download Execution time: 0.008 seconds						
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED	
1	1	1	Savings	15.713120551	6/22/2026, 6:45:29	
2	2	2	Checking	2500	6/22/2026, 6:45:29	

Step 2: Scenario 2

Problem Statement

Update employee salaries by providing a department name and bonus percentage as input parameters.

Explanation

This stored procedure increases the salaries of employees belonging to a specific department. It accepts the department name and bonus percentage as parameters and calculates the revised salary by applying the given bonus. After updating the records, the transaction is committed to save the changes permanently in the Employees table.

```
1  CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (  
2  |    p_department IN VARCHAR2,  
3  |    p_bonus IN NUMBER  
4  | )  
5  AS  
6  BEGIN  
7  |  
8  |     UPDATE Employees  
9  |     SET Salary = Salary + (Salary * p_bonus / 100)  
10 |     WHERE Department = p_department;  
11 |  
12 |     COMMIT;  
13 |  
14 END;  
15 /  
16 BEGIN  
17 |     UpdateEmployeeBonus ('IT',10);  
18 END;  
19 /  
20 SELECT * FROM Employees;
```

Output:

Query result Script output DBMS output Explain Plan SQL history						
  Download ▾ Execution time: 0.012 seconds						
		NAME	POSITION	SALARY	DEPARTMENT	HIREDATE
1	1	Alice Johnson	Manager	70000	HR	6/15/2015, 12:00:00
2	2	Bob Brown	Developer	96630.6	IT	3/20/2017, 12:00:00

```
SQL> BEGIN
      UpdateEmployeeBonus('IT',10);
    END;
```

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008

Step 3: Scenario 3

Problem Statement

Transfer a specified amount from one account to another only if sufficient balance is available in the source account.

Explanation

This stored procedure enables secure fund transfers between customer accounts. It first checks whether the source account contains enough balance to complete the transaction. If sufficient funds are available, the amount is deducted from the sender's account and credited to the receiver's account. Otherwise, the transfer is cancelled and a message is displayed. The transaction is committed only after a successful transfer.

```

1  CREATE OR REPLACE PROCEDURE TransferFunds(
2      p_from_account IN NUMBER,
3      p_to_account IN NUMBER,
4      p_amount IN NUMBER
5  )
6  AS
7      v_balance NUMBER;
8  BEGIN
9
10     SELECT Balance
11     INTO v_balance
12     FROM Accounts
13     WHERE AccountID = p_from_account;
14
15     IF v_balance >= p_amount THEN
16
17         UPDATE Accounts
18         SET Balance = Balance - p_amount
19         WHERE AccountID = p_from_account;
20
21         UPDATE Accounts
22         SET Balance = Balance + p_amount
23         WHERE AccountID = p_to_account;
24
25         COMMIT;
26
27     END IF;
28
29
30
31
32
33
34
35

```

```

28
29  END;
30  /
31  BEGIN
32      TransferFunds(1,2,5);
33  END;
34  /
35  SELECT * FROM Accounts;

```

Output:


```
SQL> BEGIN
      TransferFunds(1,2,5);
      END;
```

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014